



PlaybookPulse

Agentic Security Playbook Compliance Verifier

Product Requirements Document

24-Hour Hackathon | 3-Person Team | Python Stack

1. Executive Summary

PlaybookPulse is an agentic AI tool purpose-built for DevSecOps teams. It answers a question that no existing tool answers:

"Did we actually follow our security incident playbook, and can we prove it to an auditor?"

After a security incident, PlaybookPulse ingests the official incident response playbook alongside the actual incident trail — Slack threads, Jira tickets, and GitHub activity — then uses a multi-agent pipeline to cross-reference them step by step. It produces a color-coded deviation report, maps each gap to the relevant compliance control (NIST, SOC 2, ISO 27001), generates an auditor-ready PDF evidence package, and opens a GitHub PR to update the playbook with lessons learned.

No existing tool sits at this exact intersection. Incident management tools (PagerDuty, Blameless) track what happened. Compliance tools (Drata, Sprinto) monitor control status. PlaybookPulse bridges them: it measures the delta between intended and actual security procedure and generates proof.

2. The Problem

2.1 The Core Gap

Security incident response playbooks are written once — usually for a compliance audit — and then left to rot. When a real incident occurs, teams improvise, skip steps, and adapt on the fly. This is entirely normal and often necessary. But it creates two compounding problems:

- The playbook is never updated with what was actually learned, so the next on-call engineer faces the same stale documentation.
- There is no automatic record of where the team deviated from the playbook, which means auditors get verbal assurances rather than evidence.

2.2 The Compliance Risk

Frameworks like NIST SP 800-61, SOC 2 CC7.3, ISO 27001 A.16, GDPR Article 33, and DORA all require formalised, documented incident response. When auditors ask "show me how you handled your last security incident," most teams produce a Notion postmortem written three days after the fact — not a step-by-step adherence trace.

This creates two concrete risks: failed audits due to inadequate evidence, and playbooks that never improve because nobody is tracking where they broke down.

2.3 Why This Is Unsaturated

Tool Category	What They Do	What They Miss
---------------	--------------	----------------

PagerDuty, Blameless, Rootly	Track incidents, postmortems	Playbook adherence, compliance proof
Drata, Sprinto, Vanta	Monitor compliance controls continuously	Per-incident procedural adherence
SlothOps (similar hackathon project)	AI fixes broken code after incidents	Knowledge gaps, compliance evidence
PlaybookPulse 	Measures playbook adherence, generates compliance evidence	Nothing — this is the gap

3. Solution Architecture

3.1 High-Level Flow

PlaybookPulse operates in two phases triggered by a single CLI command or Slack slash command:

1. Ingest Phase — Pull the playbook, Slack thread, Jira ticket, and GitHub activity for the incident window.
2. Analysis Phase — Three agents run in parallel, then an orchestrator synthesizes findings into structured output.
3. Output Phase — Compliance evidence PDF is generated, GitHub PR is opened, and the dashboard updates live.

3.2 Agent Architecture

Agent	Input	Output	Tools Used
Playbook Parser	Markdown playbook file	JSON array of numbered steps with expected timelines	Claude API + tool_use
Incident Trail Agent	Slack channel ID, Jira ticket, time range	Chronological event log with actor + timestamp	Slack API, Jira REST
Adherence Checker	Parsed steps + event log	Per-step status: FOLLOWED / DELAYED / MISSED + explanation	Claude API (LLM compare)
Compliance Mapper	Deviation list	Each gap mapped to NIST / SOC2 / ISO control IDs	Claude API + control DB
Orchestrator	All agent outputs	Final structured JSON report + playbook diff hunks	Claude API

3.3 Output Artifacts

- Compliance Evidence PDF — Audit-ready document with incident timeline, step-by-step adherence trace, and control mapping table. Generated using reportlab.
- GitHub PR — Specific line-level edits to the security playbook reflecting what was missed or should be added. Opens automatically using PyGithub.
- Live Dashboard — React UI showing agent steps in real time, color-coded adherence view, and the compliance control panel.
- Slack Summary — Posted back to the incident channel with a brief deviation summary and link to the full PDF.

4. Technical Specifications

4.1 Tech Stack

Layer	Technology	Why
AI Orchestration	Claude API (claude-sonnet-4-6) with tool_use	Multi-step reasoning, structured JSON output
Agent Backend	Python 3.11 + FastAPI	Team's primary language, fast async support
Integrations	Slack Bolt SDK, Jira REST API, PyGithub	All have free tiers, Python-native SDKs
PDF Generation	reportlab or weasyprint	The auditor-ready deliverable, the demo wow moment
Dashboard / UI	React + Tailwind + WebSocket feed	Live agent steps visible to judges during demo

4.2 Key Tool Definitions (Claude tool_use)

The agent backend exposes these Python functions as Claude tools:

- `parse_playbook(md_path: str) → List[PlaybookStep]` — Extracts numbered steps, expected timelines, responsible roles, and compliance framework references from a markdown playbook.
- `fetch_slack_thread(channel_id: str, incident_start: str, incident_end: str) → List[SlackMessage]` — Returns all messages in the incident channel within the time window, with actor usernames and timestamps.
- `fetch_jira_ticket(ticket_id: str) → JiraTicket` — Returns ticket status history, comment thread, and assignee changes with timestamps.
- `fetch_github_activity(repo: str, start: str, end: str) → List[GitEvent]` — Returns commits, PRs, and review events during the incident window.
- `map_to_compliance(deviation: str, frameworks: List[str]) → List[ControlMapping]` — Maps a described deviation to specific control IDs across NIST SP 800-61, SOC 2, and ISO 27001.
- `open_github_pr(repo: str, branch: str, diff_hunks: List[str], description: str) → PRUrl` — Opens a pull request with the playbook patch.

5. 18-Hour Build Plan

5.1 Team Responsibilities

Who	Role	Key Deliverables	Hours
Person 1	Agent Backend	Multi-agent orchestrator (Python), all tool definitions, Claude API integration, adherence checker logic, compliance mapper with control ID database	~10 hrs
Person 2	Integrations	Slack Bolt bot with /playbookpulse slash command, Jira REST client, PyGithub PR automation, fixture data loader for demo, PDF evidence report (reportlab)	~10 hrs
Person 3	UI + Demo	React dashboard with WebSocket agent feed, color-coded adherence view (☑ / ⚠ / ✖), compliance control panel, demo script, slide deck	~8 hrs

5.2 Hour-by-Hour Timeline

Time Block	Milestone
Hrs 0–2	Repo setup, fixture data (sample playbook + Slack thread + Jira ticket), Claude tool scaffolding, shared JSON schema agreed
Hrs 2–6	P1: Playbook parser + Adherence Checker working end-to-end on fixtures. P2: Slack bot responds to /playbookpulse. P3: Dashboard skeleton with WebSocket connected.
Hrs 6–10	P1: Compliance Mapper + Orchestrator complete. P2: PyGithub PR opens on real repo. P3: Adherence color grid working on dashboard.
Hrs 10–14	Full pipeline integration. P2: PDF evidence report generated. P3: Compliance panel populated. Full demo run-through.
Hrs 14–16	Bug fixes, polish, edge case handling. P3: Slide deck complete. Three clean demo runs with timing.
Hrs 16–18	Buffer: rehearse pitch, stress-test fixtures, ensure PDF and GitHub PR work reliably under demo conditions.

6. Demo Flow & Pitch Script

6.1 Pre-loaded Fixture: The Scenario

A team responded to a credential leak incident last week. Their official playbook had 7 steps. The demo uses a pre-crafted Slack thread (15 messages), a Jira ticket (3 status changes, 4 comments), and 2 GitHub commits from the incident window — all fixtures loaded locally, no live API dependency.

The playbook says Step 4 (rotate affected credentials) should happen within 30 minutes of detection. In the fixture, it happened 2 hours later. Step 6 (notify legal) was skipped entirely. These two deviations are the demo's centerpiece.

6.2 Step-by-Step Demo (Target: 4 Minutes)

#	Action	What Judges See
1	Run <code>/playbookpulse #sec-incident-dec-2024</code>	Slack bot responds: "PlaybookPulse is analyzing this incident..." — sets the scene
2	Switch to dashboard	Three agent cards appear: Playbook Parser, Incident Trail Reader, Adherence Checker — all running in real time via WebSocket
3	Agents complete	Color-coded grid appears: Steps 1-3 ☒ , Step 4 ☐ DELAYED (2hr vs 30min), Step 5 ☒ , Step 6 ☒ MISSED, Step 7 ☒
4	Click Step 4	Panel shows: 'Expected by 14:30. Credential rotation PR merged at 16:47. Maps to NIST SP 800-61 §3.3.1, SOC 2 CC7.3'
5	Show PDF download	One-click generates the auditor evidence package — 4 pages, titled, timestamped, with the full control mapping table. This is the wow moment.
6	Show GitHub PR	Repo shows an auto-opened PR: '+Step 4a: Set pager alert if credential rotation exceeds 45 minutes.' — two lines, clean diff

6.3 One-Line Pitch

"PlaybookPulse tells you whether your team actually followed your security playbook during an incident — and hands you an auditor-ready PDF that proves it."

7. Risks & Mitigations

Risk	Likelihood	Mitigation
Live API rate limits during demo	High	Use pre-loaded fixture data exclusively for demo. Real APIs are bonus integrations.

LLM output non-deterministic — different run, different step classifications	Medium	Constrain with strict JSON schema in Claude system prompt. Validate against schema before rendering.
PDF generation slow (>10s)	Low	Pre-generate the PDF during the 2-min agent analysis window; surface it at the right moment in the demo flow.
GitHub PR conflicts during demo	Low	Use a dedicated demo repo with a clean branch. Reset before each demo run.
WebSocket dashboard drops during demo	Medium	Add a static fallback view that shows the last-known agent state from localStorage.

8. MVP vs. Stretch Goals

MVP (Must ship in 18 hrs)

- Playbook parser that handles markdown with numbered steps
- Adherence Checker agent producing FOLLOWED / DELAYED / MISSED classification per step
- Compliance Mapper covering NIST SP 800-61 and SOC 2 CC7.x controls
- PDF evidence report with incident summary + control mapping table
- GitHub PR auto-opened with playbook patch
- React dashboard with live agent step feed and color-coded adherence grid
- Slack bot that triggers the pipeline via slash command

Stretch Goals (if time allows)

- Live Jira API integration (replace fixture with real ticket pull)
- ISO 27001 A.16 control mapping added to the compliance mapper
- Proactive Staleness Scanner — cron job that flags playbooks not updated in 90+ days with recent incident correlation
- Multi-framework comparison — show how a single deviation maps differently across NIST, SOC 2, and ISO side by side

9. Final Summary

PlaybookPulse is a focused, buildable, and genuinely novel tool. It occupies a specific gap that no current product fills — the evidence layer between what security teams are supposed to do and what they actually do.

The three-sentence judge pitch: Security teams write incident response playbooks for auditors, then improvise during actual incidents. No tool measures the gap or generates proof of what happened. PlaybookPulse does both — and hands you a PDF you can show an auditor.

The build is scoped tightly enough for 18 hours: the agent pipeline is the core (10 hrs), the integrations are well-documented SDKs (10 hrs), and the UI is a single dashboard with a WebSocket feed (8 hrs). The fixture-first approach eliminates live API risk during the demo entirely.

The differentiator from SlothOps is clear and non-overlapping: SlothOps fixes code. PlaybookPulse proves procedure. Different problem, different output, different buyer.

Project Name	PlaybookPulse
Category	DevSecOps / Compliance Automation
Target User	Security engineers, DevSecOps leads, compliance teams at SOC 2 / ISO 27001 certified companies
Primary Stack	Python 3.11, FastAPI, Claude API, Slack Bolt, PyGithub, reportlab, React
External APIs	Anthropic Claude, Slack, GitHub (all free tier sufficient)
Compliance Frameworks	NIST SP 800-61, SOC 2 CC7.x, ISO 27001 A.16 (stretch)
Demo Duration	4 minutes end-to-end
Key Wow Moment	The auto-generated auditor-grade PDF evidence package